INSIDE STLC ACADEMY

QA LEADERSHIP ACADEMY

Quality Risk Workshop Template

Run a focused, cross-functional session that surfaces the risks worth testing, rates them by likelihood and impact, and agrees a proportionate response.

Purpose

Risk-based testing depends on knowing what the risks actually are — and the people who know are rarely all in QA. A quality risk workshop brings developers, product and QA together for a short, structured session to surface the ways a product or release could fail, rate each risk by likelihood and impact, and agree a proportionate response. Its output is a prioritised risk register that tells you where to concentrate testing effort and, just as importantly, where not to. It also builds shared ownership: risks the whole room named are risks the whole room helps mitigate.

When to use it

- At the start of a significant feature, release or programme, to shape the test approach.
- When testing effort feels spread thin and unfocused and you need to concentrate it on what matters.
- When QA and Product disagree about how much testing is "enough" — the workshop makes the risk basis explicit.
- After an incident, to reassess risks and check what the current approach missed.

Instructions

Keep it to 60–90 minutes with the right people (a developer, a product owner and a tester at minimum). Run it in four steps: (1) briefly frame the scope and what "failure" would mean to the business; (2) brainstorm risks openly — how could this go wrong, what are we assuming, what has hurt us before; (3) rate each risk for likelihood and impact using the scale below and multiply for a priority score; (4) agree a response for each — mitigate through testing, mitigate another way, monitor, or accept. Capture everything in the register. Rate impact from the customer’s and the business’s point of view, not from how hard it is to test.

Rating	Likelihood	Impact
1 — Low	Unlikely; would need an unusual combination of events	Minor annoyance; easy workaround; little business consequence
2 — Medium	Could plausibly happen	Noticeable disruption; affects some users or revenue

Rating	Likelihood	Impact
3 — High	Likely given how the change works or past experience	Severe: data loss, payment failure, outage, reputational or SLA breach

Priority = Likelihood x Impact (1–9). Responses: Mitigate (test) / Mitigate (other) / Monitor / Accept.

Worked example

Output from a risk workshop for a Northstar Digital release touching the public API and the legacy billing service.

Risk	Likelihood	Impact	Priority	Response
Breaking change to the public API breaks a B2B partner integration	2	3	6	Mitigate (test): contract tests + partner sandbox verification before release.
Refund path in legacy billing miscalculates amounts	2	3	6	Mitigate (test): targeted exploratory + data-driven checks on refund edge cases.
Unstable staging masks a real defect during testing	3	2	6	Mitigate (other): run critical-path tests in an isolated environment with managed data.
Mobile app shows stale data after the API change	2	2	4	Mitigate (test): cross-client checks on the changed endpoints.
Flaky automated checks cause a false "green" at the gate	3	2	6	Mitigate (other): use only the curated stable suite for the gate; quarantine flaky tests.
Copy/label wording is inconsistent on a settings screen	2	1	2	Monitor: note it; not worth dedicated test effort this release.
A never-used legacy report endpoint fails	1	1	1	Accept: out of scope; document the decision.

The register concentrates effort on the six-scoring risks around the API contract, refunds, environment reliability and gate trust, while explicitly accepting or monitoring the low scorers — so nobody spends a day perfecting a settings-screen label while the refund path goes lightly tested.

Blank reusable version

Risk	Likelihood (1–3)	Impact (1–3)	Priority	Response

Priority = Likelihood x Impact. Response = Mitigate (test) / Mitigate (other) / Monitor / Accept.

Common mistakes

COMMON MISTAKES

Running the workshop with only QA in the room, so the risks developers and product know about never surface. Rating impact by how hard something is to test rather than by consequence to the customer and business. Turning it into a marathon that exhausts everyone — keep it tight. Producing a register and then never using it to shape the actual test effort, so it becomes theatre. And treating "accept" as a dirty word: consciously accepting a low risk is good risk management, not negligence — the mistake is accepting risks silently and by accident.

How to interpret the results

- High-priority risks (typically 6 and 9) are where your testing effort concentrates. If your test plan does not visibly address the top risks, rework the plan.
- A "mitigate (other)" response is a legitimate outcome — sometimes the right fix is a feature flag, better monitoring or a design change, not more testing. Not every risk is a testing problem.
- Low-priority risks marked "accept" or "monitor" are where you deliberately save effort. Being able to point to them explains why you did not test everything.
- If everything scores high, either the change is genuinely dangerous (slow down and talk to Product) or the room is rating defensively. Re-anchor against the scale.
- Keep the register live through the work. Risks that materialise, or new ones that appear, should update it — and feed real incidents back into how you rate similar risks next time.

INSIDE STLC PRO TIP

The register is your best answer to "why did you not test that?" and "why are you spending so long testing this?" alike. When Product pushes on a date, walk them through the top risks and what happens if you skip mitigating them — the conversation shifts from "QA is slow" to "which of these risks are we as a business willing to accept?" That is the question a QA leader should always be helping the business answer.